

Raport Tehnic - NetDiag

Coşulean Dima, anul 2, grupa B2

Universitatea „Alexandru Ioan Cuza”, Iaşi
Facultatea de Informatică
<https://info.uaic.ro/>

Cuprins

Raport Tehnic - NetDiag.....	1
<i>Coșulean Dima, anul 2, grupa B2</i>	
1 Viziune și obiective.....	2
2 Tehnologii folosite.....	2
3 Structura aplicației.....	3
4 Protocolul de comunicare	6
5 Scenarii de utilizare	6
6 Concluzii	8
7 Bibliografie.....	8

1 Viziune şi obiective

Proiectul urmăreşte dezvoltarea unei aplicaţii care oferă o vizualizare dinamică şi în timp real a traseului reţelei dintre un sistem sursă şi o destinaţie IP. Aplicaţia îşi propune să ofere utilizatorului o perspectivă completă asupra infrastructurii de reţea intermediare, fiind capabilă să monitorizeze continuu ruta pachetelor şi să semnaleze eventualele probleme de conectivitate.

Am ales acest proiect datorită relevanţei sale în domeniul reţelelor. Consider că o astfel de aplicaţie este extrem de utilă atât pentru identificarea problemelor de infrastructură, cât şi în monitorizarea performanţei unei rute de comunicaţie. NetDiag poate ajuta la depistarea rapidă a nodurilor care nu mai răspund, la analizarea timpilor de latenţă şi la înţelegerea modului în care traficul circulă prin reţea. Fiind foarte valoros atât în contexte educaţionale şi de cercetare, cât şi în administrarea reţelelor din viaţa reală, unde o diagnosticare corectă şi rapidă poate preveni sau rezolva probleme critice.

2 Tehnologii folosite

Pentru realizarea acestui proiect am folosit **socketuri raw**, deoarece aplicaţia trebuie să genereze şi să prelucreze pachete **ICMP** în mod direct. Această abordare este necesară pentru implementarea logicii de tip *traceroute*, unde aplicaţia controlează manual câmpuri precum **TTL** şi codurile ICMP, astfel încât să poată determina ruta completă până la destinaţie şi latenţa fiecărui hop. Întrucât ICMP funcţionează peste IP şi nu este encapsulat în TCP sau UDP, utilizarea unui socket raw este singura soluţie care permite accesul direct la nivelul reţelei şi la toate detaliile necesare diagnosticării.

Pentru comunicarea dintre client şi server am utilizat un protocol bazat pe mesaje delimitate, transmis printr-o conexiune TCP. Alegerea TCP se justifică prin faptul că partea de control trebuie să fie fiabilă şi fără pierderi. Dacă o comandă trimisă de client este pierdută, serverul poate rula cu parametri greşiţi sau incompleţi, ceea ce ar compromite rezultatele măsurărilor. Prin urmare, TCP garantează atât livrarea, cât şi ordinea corectă a mesajelor de configurare.

Ceea ce priveşte comunicarea concurentă, am implementat un model hibrid care combină mecanismul de multiplexare I/O prin *select()* cu cel de threading pentru operaţii lungi. Primitiva *select()* monitorizează non-blocant mai mulţi clienţi simultan, fiind eficientă pentru comunicare şi comenzi rapide. Deoarece operaţia de traceroute poate dura minute întregi, am utilizat *pthread_create()* pentru a executa traceroute-ul în thread-uri separate, permiţând serverului să rămână responsiv şi să proceseze alte comenzi (STOP, RESTART, etc.) în paralel. În contextul NetDiag, această soluţie este optimă, deoarece asigură că serverul nu se blochează în timpul traceroute-ului şi că va trimite răspunsurile în timp real către client, fără overhead-ul semnificativ al proceselor multiple generate prin *fork()*.

3 Structura aplicației

Aplicația constă dintr-un server concurent, care gestionează procesarea comenzilor și execuția acestora și un client CLI (*Command Line Interface*), ce este responsabil de interacțiunea cu utilizatorul, trimiterea comenzilor către server și afișarea răspunsurilor pe măsura ce vin. Comunicarea este stabilită în subprogramele „main” *Server_NetDiag* și respectiv *Client_NetDiag* ale serverului și clientului. Schema comunicării este ilustrată în 1.

Conexiunea se stabilește printr-un canal TCP, iar răspunsurile de la server sunt precedate de lungimea în bytes. După stabilirea conexiunii clientul trimite comenzi către server, iar serverul rămâne permanent activ, gata de a procesa simultan mai mulți clienți.

După recepționarea unui mesaj de la client, serverul îl validează prin intermediul funcției *parse_Command* din subprogramul *command*, aici se verifică corectitudinea comenzii, se extrag argumentele (*în cazul în care acestea fac parte din comandă*) și se mapază într-o structură internă de tip *Command*. Dacă formatul este corect și toate argumentele necesare sunt prezente, comanda este executată de către funcția *command_Executor*, unde în funcție de tipul acesteia, serverul poate realiza operații imediate (*ex. oprirea unei monitorizări, setarea unui parametru, etc.*) sau operații mai complexe (*ex. inițierea unui proces de diagnosticare*), pentru aceste cazuri, serverul creează un nou fir de execuție (**thread**), astfel încât execuția traceroute-ului să nu blocheze procesarea altor comenzi. Pentru utilizarea mai eficientă a resurselor, firele de execuție sunt create doar în momentul în care serverul primește de la client o comandă de lansare a diagnosticării (*start sau report*), în acest moment se setează un flag ce arată că pentru clientul dat există deja un fir de execuție dedicat ce se află într-o stare de așteptare sau execută o diagnoză cerută. În timpul execuției, thread-ul dedicat trimite periodic rezultatele intermediare către client, folosind același protocol de mesaje prefixate de lungime, astfel permițând afișarea rutei și informațiilor specifice în timp real pe măsura execuției.

Funcționalitatea aplicației poate fi urmărită în schema de la 2

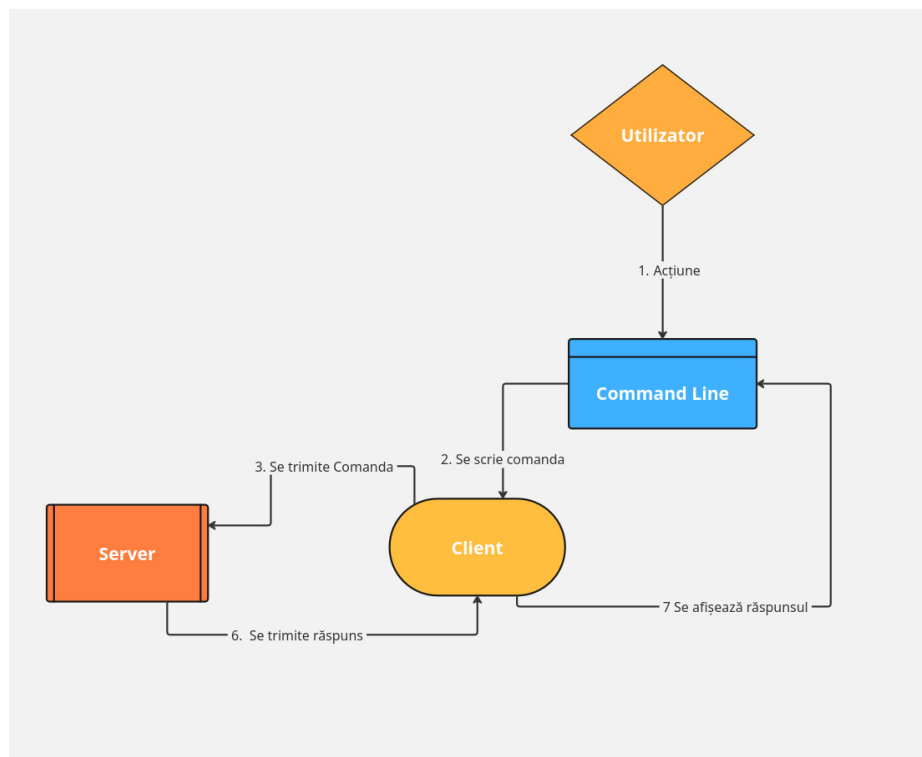
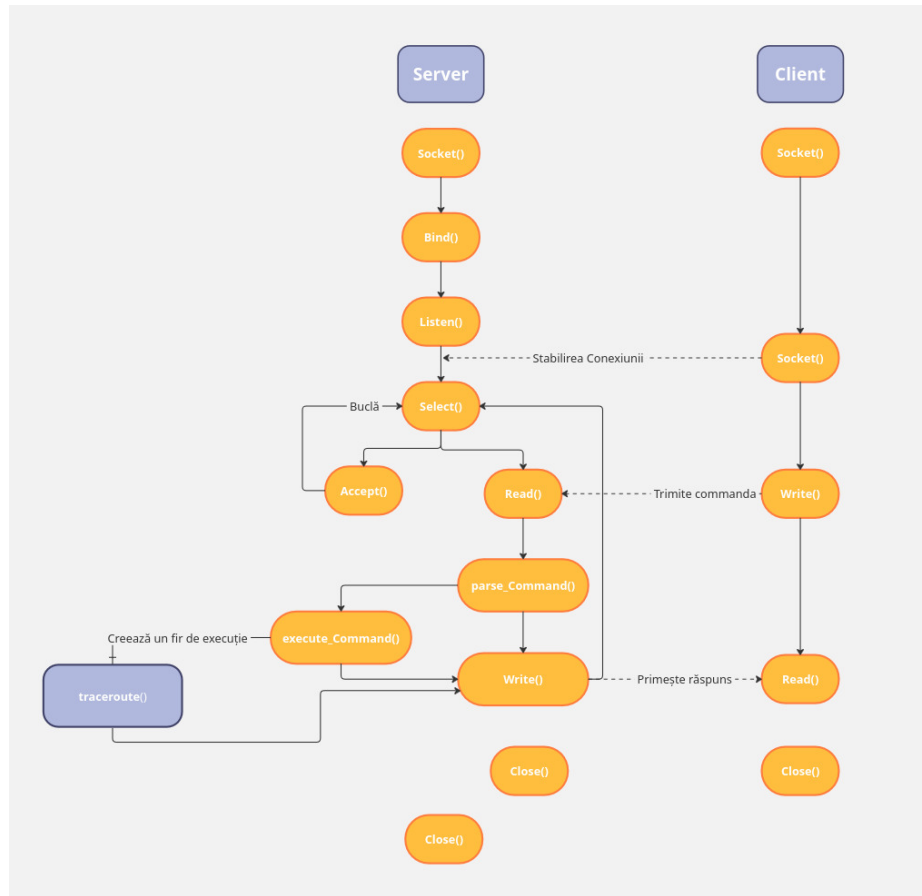


Figura 1. Structura aplicației: NetDiag

**Figura 2.** Comunicarea Client Server

4 Protocolul de comunicare

Protocolul la nivelul aplicație este de tip mesaje cerere - răspuns, construit peste o conexiune fiabilă **TCP**. Problema fiabilității comunicării este rezolvată cu modelul răspunsului confirmat în server și modelul cerere/răspuns în client. Explicit, clientul trimite o cerere, serverul confirmă primirea cererii și apoi trimite răspuns, în timp ce clientul așteaptă răspunsul. Pentru a evita pierderea datelor, răspunsurile au o structură fixată, prefixate de lungimea mesajului ce urmează să fie trimis.

Utilizatorul interacționează numai cu programul client, prin intermediul liniei de comandă unde poate introduce comenzi pentru setarea parametrilor, controlul traceroute-ului, afișarea unor rezultate sau administrative.

După recepție serverul parsează comanda prin intermediul funcției *Parse_Command()* care identifică tipul comenzii, extrage argumentele și construiește o structură internă de tip *COMMAND*. În cazul comenzilor de control rapide (stop, quit, set dest <IP>), serverul procesează imediat cererea și trimite un răspuns. Pentru comenzile care declanșează operații de durată (start, report), serverul creează un thread dedicat care transmite periodic mesaje către client, în același format prefixat cu lungime, asigurând astfel o actualizare în timp real.

5 Scenarii de utilizare

Aplicația NetDiag reprezintă un instrument pentru înțelegerea modului în care funcționează traseele pachetelor într-o rețea IP și pentru depistarea nodurilor problematice, fiind util atât în scop didactic, cât și în viața reală. Funcționează similar instrumentelor reale de diagnosticare, precum *traceroute* sau *mtr*.

Cu toate acestea, aplicația pornește un traceroute doar la cererea utilizatorului, iar regulile de procesare sunt controlate manual, fără optimizările și mecanismele avansate puse la dispoziție de către instrumentele profesionale.

5.1 Execuție cu succes

1. Pornirea unui traceroute valid către o destinație setată explicit

Utilizatorul introduce în interfața client comanda *set dest <IP>* urmată de comanda *start*. Serverul validează adresa IP, creează un fir de execuție dedicat și inițiază procesul traceroute. Clientul primește periodic informații despre fiecare hop, pe care le afișează în timp real. Execuția continuă până când utilizatorul trimite comanda *stop*. În cazul în care utilizatorul trimite *stop* fără a modifica parametrii, traceroute-ul se execută cu valorile implicite: destinație localhost, TTL maxim 30, interval de 1 secundă și 3 probe per TTL.

2. Ajustarea parametrilor implicați pentru optimizarea analizei

Utilizatorul rulează comenzi precum *set maxttl 15*, *set interval 2*, *set probes 5*, apoi execută *start*. Serverul validează fiecare parametru, actualizează configurația firului de execuție și rulează traceroute cu noile valori. Clientul primește un flux

de rezultate bine structurat, optimizat pentru analiza rețelelor cu latențe ridicate sau cu rute instabile. Finalizarea execuției se realizează după ce clientul trimite comanda *stop*.

3. Generarea unui raport cu comanda *report*

La introducerea comenzii *report*, serverul inițiază automat o serie de execuții ale traceroute-ului. Pentru această operație este creat un fir de execuție separat, astfel încât serverul să își păstreze capacitatea de a procesa în paralel alte comenzi de la client. În fiecare rundă sunt colectate informații precum timpii de răspundere individuali, încercările cu succes și cele în care router-ul nu a răspuns. După finalizarea întregului set de rulări se creează un raport ce se trimite către client iar apoi se afișează pe ecran.

Numărul de rulări pentru crearea raportului poate fi setat și de către utilizator cu comanda *set cycle <nr>*.

5.2 Execuție cu eșec

1. Pornirea traceroute-ului fără setarea destinației și cu parametri invalizi

În situația în care utilizatorul încearcă să configureze parametrii traceroute-ului folosind date invalide, de exemplu o adresă IP incorectă, un maxTTL în afara intervalului permis sau un număr negativ de probe, serverul detectează automat această neconcordanță în etapa de parsare (în funcția *Parse_Command*). Parametrii sunt validați strict înainte de a fi salvați în structura *COMMAND*, iar dacă unul dintre aceștia nu respectă constrângerile impuse, serverul nu permite modificarea parametrului.

În loc să modifice parametrii și să poată rula diagnosticarea cu date eronate, serverul trimite clientului un mesaj explicit de eroare, indicând exact care parametru este invalid și, domeniul de valori acceptat.

2. Deconectarea clientului în timpul execuției

Dacă în timp ce traceroute-ul rulează, clientul se închide brusc. Serverul detectează în *select()* că socketul clientului a devenit invalid. Thread-ul asociat execuției este notificat să se oprească, iar resursele sunt eliberate. Deoarece conexiunea TCP nu mai este activă, serverul nu poate trimite mesajul final, iar execuția este clasificată ca eșuată.

3. Utilizatorul încearcă să ruleze un traceroute în timp ce alt traceroute este deja activ

Dacă există deja un proces de diagnosticare lansat de client, iar acesta mai trimite încă o comandă *start* serverul detectează un conflict. Pentru a evita suprascrierea datelor sau alocări haotice de resurse, serverul refuză comanda și-i transmite clientului un mesaj corespunzător.

6 Concluzii

Aplicația realizează funcționalitățile esențiale ale unui instrument de diagnosticare în rețea, având o logică proprie de traceroute implementată manual prin socketuri raw. Deși soluția propusă funcționează corect, există numeroase direcții prin care aplicația ar putea fi extinsă, optimizată și adusă mai aproape de un instrument profesional real.

Un aspect semnificativ ce poate fi îmbunătățit este suportul extins pentru formatele de adresare. În prezent, aplicația funcționează exclusiv cu adrese IPv4 furnizate explicit, însă o versiune avansată ar putea permite și setarea destinațiilor precum *"google.com"* sau altele, suport complet pentru formatul IPv6. Aceste extinderi ar crește compatibilitatea aplicației cu infrastructurile moderne și ar permite diagnosticarea corectă a rețelelor hibride IPv4/IPv6, reprezentative pentru Internetul actual.

În plus, aplicația ar putea fi extinsă cu o interfață grafică dedicată, care să permită utilizatorului final să configureze parametrii traceroute-ului, să vizualizeze grafic evoluția latențelor și să analizeze rapoartele într-o manieră mult mai intuitivă. O astfel de interfață ar transforma NetDiag într-un instrument accesibil și utilizabil și de persoane fără experiență directă în lucrul cu terminalul.

7 Bibliografie

1. Site-ul disciplinei Rețele de Calculatoare "<https://edu.info.uaic.ro/computer-networks/index.php>"
2. Overleaf, instrument pentru crearea și editarea documentului în Latex "<https://www.overleaf.com/>"
3. Miro, instrument entru crearea diagramelor "<https://miro.com/app/board/uXjVGe8xjtc=/>"
4. Comanda mtr - Linux man page = mtr(8) "<https://linux.die.net/man/8/mtr>" accesat ultima dată pe data de 03.12.2025
5. Socket raw - Linux man page = raw(7) "<https://man7.org/linux/man-pages/man7/raw.7.html>" accesat ultima dată pe data de 06.12.2025
6. stackoverflow, "<https://stackoverflow.com/questions/14774668/what-is-raw-socket-in-socket-programming>", accesat ultima dată pe 06.12.2025
7. stackoverflow, "<https://stackoverflow.com/question/55218931/calculating-checksum-for-icmp-echo-request-in-python>", accesat ultima dată la data de 08.12.2025
8. "<https://habr.com/ru/companies/rvuds/articles/516266/>" accesat ultima dată la data de 08.12.2025